Hardware Optimization: Bit Shifting & Masking

Trading Space and Speed in the CPU

Bernal Manzanilla Saavedra

Concordia University – IT110

April 10, 2026

The Need for Speed

The Problem: Inside the CPU, the Arithmetic Logic Unit (ALU) is responsible for math. Adding is fast. But Multiplying and Dividing are complex operations that take several hardware clock cycles to finish.

The Need for Speed

The Problem: Inside the CPU, the Arithmetic Logic Unit (ALU) is responsible for math. Adding is fast. But Multiplying and Dividing are complex operations that take several hardware clock cycles to finish.

The Solution: If we need to multiply or divide by powers of 2 (like 2, 4, 8, 16...), we can cheat. Instead of doing "math," we can just physically slide the binary 1s and 0s to the left or right inside the register.

A physical "Bit Shift" takes exactly **one single clock cycle**. It is blindingly fast.

Logical Shift Left (LSL)

A **Logical Shift Left** moves all bits to the left by a specified number of spaces. The empty spaces on the right are filled with 0s.

The Mathematical Rule: Shifting left by n bits multiplies the number by 2^n.

Logical Shift Left (LSL)

A **Logical Shift Left** moves all bits to the left by a specified number of spaces. The empty spaces on the right are filled with 0s.

The Mathematical Rule: Shifting left by n bits multiplies the number by 2^n.

Example: Multiply by 2

Start with Decimal 5: 0000 0101
Shift Left by 1: 0000 1010
The new value is 10! $(5 \times 2^1 = 10)$

Example: Multiply by 8

Start with Decimal 5: 0000 0101
Shift Left by 3: 0010 1000
The new value is 40! $(5 \times 2^3 = 40)$

Logical Shift Right (LSR)

A **Logical Shift Right** moves all bits to the right. The bits that fall off the right edge are deleted (dropping the remainder). The empty spaces on the left are filled with 0s.

The Mathematical Rule: Shifting right by n bits divides the number by 2^n.

Logical Shift Right (LSR)

A **Logical Shift Right** moves all bits to the right. The bits that fall off the right edge are deleted (dropping the remainder). The empty spaces on the left are filled with 0s.

The Mathematical Rule: Shifting right by n bits divides the number by 2^n.

Example: Divide by 4

Start with Decimal 24: 0001 1000
Shift Right by 2: 0000 0110
The new value is 6! $(24/2^2 = 6)$

The Danger of Signed Numbers

Logical Shifts work perfectly for Unsigned (always positive) integers. But what if we are using Two's Complement signed integers?

Remember, in Two's Complement, the far-left bit is the **Sign Bit** (1 means negative, 0 means positive).

The Danger of Signed Numbers

Logical Shifts work perfectly for Unsigned (always positive) integers. But what if we are using Two's Complement signed integers?

Remember, in Two's Complement, the far-left bit is the **Sign Bit** (1 means negative, 0 means positive).

The Logical Shift Disaster

Start with Decimal -4: `1111 1100`
Logical Shift Right: `0111 1110`
Because an LSR blindly drops a 0 into the leftmost slot, it overwrites the sign bit. Our -4 just instantly became +126! The math is ruined.

Arithmetic Shift Right (ASR)

To safely divide negative numbers, the CPU uses an **Arithmetic Shift Right (ASR)**.

An ASR shifts the bits to the right, but instead of filling the empty left slot with a 0, it **copies** whatever the sign bit currently is.

Arithmetic Shift Right (ASR)

To safely divide negative numbers, the CPU uses an **Arithmetic Shift Right (ASR)**.

An ASR shifts the bits to the right, but instead of filling the empty left slot with a 0, it **copies** whatever the sign bit currently is.

The Safe Division

Start with Decimal -4: `1111 1100`
Arithmetic Shift Right: `1111 1110`
The 1 is duplicated. The sign is preserved. The new value is exactly -2.
The math works perfectly!

Space Complexity: Bit Packing

Now we know how shifts help with **Speed**. Let's talk about **Space**.

A boolean value (True/False) only needs 1 bit of memory. But the smallest chunk of memory a computer can normally request is a full 8-bit byte. Storing one True/False flag per byte wastes 7 bits of empty space!

Space Complexity: Bit Packing

Now we know how shifts help with **Speed**. Let's talk about **Space**.

A boolean value (True/False) only needs 1 bit of memory. But the smallest chunk of memory a computer can normally request is a full 8-bit byte. Storing one True/False flag per byte wastes 7 bits of empty space!

The Solution

Programmers pack 8 different boolean flags into a single byte. To check just one specific flag without the others interfering, we use a **Bit Mask**.

The Logical AND Mask

To extract data, we apply a Mask using the logical **AND** operator.
Rule: `1 AND 1 = 1`. `Anything AND 0 = 0`.

If you want to keep a bit, put a 1 in your mask. If you want to erase a bit, put a 0 in your mask.

The Logical AND Mask

To extract data, we apply a Mask using the logical **AND** operator.
Rule: `1 AND 1 = 1`. `Anything AND 0 = 0`.

If you want to keep a bit, put a 1 in your mask. If you want to erase a bit, put a 0 in your mask.

Extracting the Lower Nibble

Target Data Register: `1011 0110`
Our Custom Mask: `0000 1111`

Result after AND: `0000 0110`

The top half is completely erased to 0, leaving only the specific data we wanted to read.

Time to Practice! Work through the exercises on your handout, then complete the Moodle Quiz.